

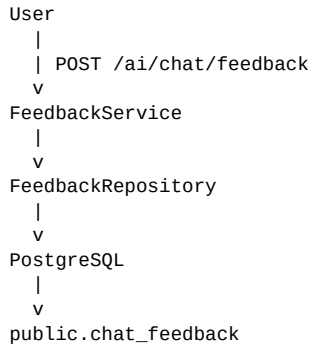
PaiseWise Task 3 – Feedback Schema

PostgreSQL feedback persistence, API contract, repository flow, and weekly analytics

1. Purpose

The Task 3 feedback component records a user's thumbs-up or thumbs-down rating for an AI-generated chat response. Feedback persistence and analytics are kept separate from the database-independent AI ChatService.

2. Architecture



The AI chat path is separate: ChatService uses JSON input, RAG, Redis conversation context, the LLM, and response validation. It does not directly use FeedbackRepository or PostgreSQL.

3. Feedback Table

The FeedbackRepository supplied for Task 3 inserts into **public.chat_feedback** using response_id, user_id, category, and feedback, and returns id, response_id, user_id, category, feedback, and created_at.

Column	Purpose
id	Unique feedback record identifier.
response_id	Identifies the AI response receiving feedback.
user_id	Identifies the user submitting feedback.
category	Category associated with the AI response.
feedback	User rating: up or down.
created_at	Time when feedback was recorded.

4. Repository Insert

```
INSERT INTO public.chat_feedback (
    response_id,
    user_id,
    category,
    feedback
)
VALUES (%s, %s, %s, %s)
RETURNING
    id,
    response_id,
    user_id,
    category,
    feedback,
    created_at;
```

The repository uses the project's get_db() context manager. A successful operation is committed; an exception is rolled back and re-raised.

5. Feedback API Request

```
POST /ai/chat/feedback
```

```
{
  "responseId": "response-123",
  "userId": "user-001",
  "category": "general_knowledge",
  "feedback": "up"
}
```

ChatFeedbackRequest validates responseId, userId, and category as non-empty strings. The feedback value is restricted to **up** or **down**.

6. Feedback API Response

```
{
  "status": "success",
  "message": "Feedback received successfully."
}
```

7. Weekly Analytics

The repository calculates feedback statistics for the last seven days, grouped by category. It returns thumbs_up, thumbs_down, and total_feedback.

```
SELECT
  category,
  COUNT(*) FILTER (
    WHERE feedback = 'up'
  ) AS thumbs_up,
  COUNT(*) FILTER (
    WHERE feedback = 'down'
  ) AS thumbs_down,
  COUNT(*) AS total_feedback
FROM public.chat_feedback
WHERE created_at >= NOW() - INTERVAL '7 days'
GROUP BY category
ORDER BY category;
```

8. Analytics Response

```
GET /ai/chat/feedback/analytics
```

```
[
  {
    "category": "general_knowledge",
    "thumbs_up": 15,
    "thumbs_down": 3,
    "total_feedback": 18
  }
]
```

9. What the Feedback Component Stores

Persisted feedback data	Not fetched by isolated ChatService
response_id	Profile information
user_id	Holdings
category	Orders
feedback	Conversation history from PostgreSQL
created_at	Other personal user data

10. End-to-End Flow

1. User sends a chat request.
2. AI service generates a responseId.
3. User selects thumbs up or thumbs down.
4. Client sends ChatFeedbackRequest.
5. FeedbackService validates the request.
6. FeedbackRepository stores the feedback in PostgreSQL.
7. Analytics reads feedback from the last 7 days.

8. Results are grouped by category and returned as JSON.

11. Manager-ready Summary

Task 3 feedback is a separate persistence component. The AI ChatService remains database-independent. When a user submits thumbs-up or thumbs-down, the feedback API sends responseld, userId, category, and the rating to FeedbackService. FeedbackRepository stores the record in PostgreSQL, and the analytics endpoint aggregates the last seven days by category.

12. Implementation Boundary

This document reflects the FeedbackRepository, FeedbackService, API schemas, and routes provided during the Task 3 implementation. Exact database constraints not shown in the supplied repository code should be taken from the project's actual migration/schema rather than inferred.